TD progressif : Valkey, cache et invalidation

Durée des TD : 125 minutes en quatre parties : connexion à Valkey (30 min), mise en cache (40 min), invalidation (30 min), puis limitation des requêtes (25 min). Avancez étape par étape et vérifiez le résultat avant de continuer. La branche `jour4-valkey-cache-documents` contient la correction des trois premières parties ; `jour4-rate-limiting-documents` contient la correction de la quatrième.

À quoi sert Valkey ?

Valkey est un serveur de données rapide qui peut servir à plusieurs besoins :

Utilisation	Exemple
Cache	Conserver temporairement une liste de documents pour la relire rapidement.
Files d'attente	Garder des tâches à traiter, comme l'analyse d'un PDF ou l'envoi d'un email. Un worker Laravel exécute ensuite la tâche.
Rate Limiting	Compter les requêtes d'un utilisateur pendant une durée donnée.
Sessions	Conserver côté serveur les informations de session d'un utilisateur connecté.
Verrous	Coordonner des traitements qui doivent éviter de modifier une même ressource simultanément.
Pub/Sub	Diffuser un événement aux applications abonnées à un canal.
Streams	Conserver une suite de messages ou d'événements à lire par des consommateurs.
Compteurs et classements	Compter les téléchargements ou afficher les documents les plus consultés.
Données géographiques	Rechercher les agences proches d'une position.

Dans notre formation, nous utilisons Valkey pour le **cache** et les **compteurs de Rate Limiting**. Les autres usages sont présentés à titre d'exemples ; ils ne sont pas tous réalisés dans ce projet. Pub/Sub diffuse aux abonnés présents ; une file de tâches et un stream répondent à d'autres besoins. La conservation des données en cas d'arrêt dépend de la configuration de persistance de Valkey.

Pour en savoir plus : présentation de Valkey et types de données.

Avant de commencer

Dans VS Code, ouvrez le dossier **Laravel** qui contient `compose.yaml` et `artisan`. Dans **Terminal > Nouveau terminal**, choisissez PowerShell :

```
git branch --show-current
git status
docker compose ps
```

À vérifier : partez de `jour3-securite-documents` ou de votre travail terminé du jour 3. Docker Desktop est démarré. PostgreSQL et Laravel sont déjà configurés, les migrations de sécurité sont réalisées et les comptes fictifs sont prêts. Utilisez uniquement la base locale de formation. Sauvegardez vos modifications avant de changer de branche ; si Git refuse, demandez au formateur et ne supprimez pas de fichiers.

Dans ce TD, ne relancez pas les migrations ni les seeders : ils ne sont pas nécessaires si le jour 3 est prêt. Ne recréez pas `.env`, ne régénérez pas `APP_KEY` et ne supprimez pas les volumes Docker. Les essais utilisent des documents et comptes fictifs.

Le projet interface peut rester sur `jour4-interface-api-securisee`. Pour voir précisément les statuts et les données, nous utilisons ici **Postman**, comme au jour 3.

Partie 1 : démarrer Valkey et le connecter à Laravel — 30 min

1. Ajouter le service Docker

Ouvrez `compose.yaml`. Sous `services:`, repérez le bloc `pgadmin:` et collez ce nouveau service **juste avant**, au même niveau d'indentation que `app:`, `db:` et `pgadmin:` :

```
  valkey:
    image: valkey/valkey:9.0.7-alpine
    # Cache temporaire : les données sont reconstruites depuis PostgreSQL.
    command: ["valkey-server", "--save", "", "--appendonly", "no"]
    healthcheck:
      test: ["CMD", "valkey-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 12
```

Dans le bloc `app:`, trouvez `depends_on:` et ajoutez les deux lignes `valkey:` et `condition:` ci-dessous. **Conservez aussi la dépendance `db` existante** :

```
    depends_on:
      db:
        condition: service_healthy
      valkey:
        condition: service_healthy
```

Enregistrez. Ce service ne publie aucun port sur Windows : Laravel le contacte sur le réseau Docker par le nom `valkey`. Ici Valkey est un cache temporaire sans persistance configurée ; après recréation du conteneur, le cache sera reconstruit. PostgreSQL et les PDF privés restent les données de référence.

À vérifier : demandez à Docker de vérifier le fichier, puis démarrez les services :

```
docker compose config --quiet
docker compose up -d
docker compose ps
docker compose exec valkey valkey-cli ping
```

La première commande ne doit pas afficher d'erreur. `valkey` doit apparaître démarré et la dernière commande doit répondre **PONG**. Le premier démarrage peut télécharger l'image.

À vous

Expliquez à votre voisin : pourquoi `valkey` est-il un service séparé de `db` ? Quelle commande a montré que Valkey répond ?

2. Installer le client PHP

Laravel utilise un client compatible Redis pour communiquer avec Valkey. Installez Predis dans le conteneur `app` :

```
docker compose exec app composer require predis/predis:^3.2 --no-interaction --no-progress
```

Composer met à jour `composer.json`, `composer.lock` et les dépendances du volume Composer de ce projet. Cette étape peut nécessiter Internet. Si elle échoue, lisez l'erreur avec le formateur avant de continuer.

À vérifier :

```
docker compose exec app composer show predis/predis
```

Le nom et une version de Predis doivent apparaître. La correction a été préparée avec la version verrouillée par son fichier `composer.lock`.

3. Configurer Laravel

Dans VS Code, ouvrez votre fichier **`.env` existant**. Modifiez seulement les lignes suivantes, sans toucher aux autres paramètres :

```
CACHE_STORE=redis
REDIS_CLIENT=predis
REDIS_HOST=valkey
REDIS_PASSWORD=null
REDIS_PORT=6379
```

`valkey` est le nom du service Docker. `redis` est le nom du pilote prévu par Laravel ; il peut parler à Valkey grâce à leur compatibilité. Le mot « Redis » dans la configuration ne signifie pas qu'un second serveur doit être installé. Si Valkey est arrêté, cette correction ne prévoit pas de bascule automatique : les opérations utilisant le cache peuvent échouer jusqu'à son redémarrage. Une écriture PostgreSQL peut avoir réussi avant une erreur d'invalidation ; vérifiez le document avant de réessayer pour éviter un doublon.

Pour qu'une **nouvelle** installation de formation ait les mêmes valeurs par défaut, appliquez les mêmes changements dans `.env.example` : remplacez `CACHE_STORE=file`, `REDIS_CLIENT=phpredis` et `REDIS_HOST=127.0.0.1` par les trois premières lignes ci-dessus. Ne copiez pas `.env` dans Git : il contient la configuration propre au poste.

Si Laravel avait une configuration mise en cache, cette commande retire seulement le fichier de configuration compilée ; elle ne vide ni PostgreSQL ni les données Valkey :

```
docker compose exec app php artisan config:clear
docker compose restart app
docker compose exec app php artisan config:show cache.default
```

À vérifier : la dernière commande affiche `redis`. Le redémarrage de `app` lui fait relire la configuration. La limitation de connexion du jour 3 utilise maintenant ce cache par défaut : ses anciens compteurs enregistrés avec le pilote précédent ne sont pas transférés.

4. Tester un aller-retour Laravel / Valkey

Ouvrez Tinker :

```
docker compose exec app php artisan tinker
```

Dans l'invite Tinker `>`, exécutez les lignes suivantes une par une :

```
\Illuminate\Support\Facades\Cache::put('td4:bonjour', 'Bonjour Valkey', 60);
\Illuminate\Support\Facades\Cache::get('td4:bonjour');
\Illuminate\Support\Facades\Cache::forget('td4:bonjour');
exit
```

À vérifier : la deuxième ligne affiche `"Bonjour Valkey"`. La troisième supprime seulement cette clé de démonstration. Si vous obtenez une erreur de connexion, vérifiez le service, `REDIS_HOST`, `REDIS_CLIENT` et le redémarrage de `app`.

À vous

Arrêtez-vous ici : que représentent `td4:bonjour`, `Bonjour Valkey` et `60` ? Quel composant a conservé la valeur pendant l'essai ?

Partie 2 : mettre en cache la liste personnelle — 40 min

5. Préparer un document dans Postman

Connectez Amina avec `POST http://localhost:8088/api/login`, en choisissant **Body > raw > JSON** et en ajoutant `Accept: application/json`. Utilisez les comptes de formation du jour 3. Dans une requête `POST http://localhost:8088/api/documents`, sélectionnez **Authorization > Bearer Token**, collez le token d'Amina et envoyez :

```
{ "title": "Document cache Amina", "category": "Formation" }
```

À vérifier : réponse **201**. Notez l'`id` du document et son `user_id` ; cet identifiant utilisateur permettra de reconnaître sa clé de cache. Les identifiants peuvent être différents de ceux montrés en cours.

6. Adapter la lecture

Ouvrez `app/Http/Controllers/DocumentController.php`. Ajoutez cette ligne parmi les autres `use` :

```
use Illuminate\Support\Facades\Cache;
```

Remplacez uniquement la méthode `index` par :

```
    public function index(Request $request): JsonResponse
    {
        $key = 'documents:user:'.$request->user()->id;
        $documents = Cache::remember($key, now()->addMinutes(5),
            fn () => $request->user()->documents()->get()->toArray());

        return response()->json($documents);
    }
```

Lisez le code : la clé contient l'identifiant de l'utilisateur connecté. `remember` cherche cette clé ; si elle manque, la fonction lit PostgreSQL et conserve le **tableau** obtenu pendant cinq minutes. `toArray()` conserve un format de données simple pour la sérialisation du cache. Laravel renvoie ensuite la réponse JSON.

7. Observer le cache

1. Dans Postman, envoyez `GET http://localhost:8088/api/documents` avec le token d'Amina.
2. Dans PowerShell, listez les clés de l'exercice :

```
docker compose exec valkey valkey-cli -n 1 --scan --pattern '*documents:user:*'
```

À vérifier : une clé se termine par `documents:user:` suivi du `user_id` noté précédemment. Laravel ajoute un préfixe avant cette partie. `-n 1` interroge la base logique utilisée par défaut pour le cache dans ce projet ; si `REDIS_CACHE_DB` a été personnalisé, demandez au formateur de vérifier sa valeur.

3. Copiez **la clé complète affichée**, préfixe compris, dans la commande suivante à la place du texte entre apostrophes :

```
docker compose exec valkey valkey-cli -n 1 TTL 'COLLER_LA_CLE_COMPLETE_ICI'
```

À vérifier : le nombre de secondes est positif et inférieur ou égal à 300. `-2` signifie que la clé n'existe plus : renvoyez le GET. `-1` signifie qu'aucune expiration n'est définie : vérifiez le code.

4. Renvoyez le même GET avec le token d'Amina. La liste reste identique et la clé existe toujours. Le temps restant ne repart pas automatiquement à 300 : la copie existante est réutilisée.
5. Connectez Karim dans Postman, puis envoyez la même requête avec **son** token. **À vérifier :** sa liste ne contient pas le document d'Amina. La commande `--scan` affiche une autre clé, se terminant par l'identifiant de Karim. Une liste vide est possible.

À vous

Avant de passer à la suite, répondez : pourquoi une seule clé `documents` pour tout le monde serait-elle dangereuse ? Si la clé expire, que fera la prochaine lecture ?

Partie 3 : invalider la liste après une écriture — 30 min

8. Ajouter l'invalidation dans le contrôleur

Après chaque modification réussie de la liste, nous supprimons **uniquement la clé de l'utilisateur concerné**. La prochaine lecture la reconstruira.

Dans `DocumentController.php`, ajoutez cette ligne **juste après** la création du document dans `store` :

```
        Cache::forget('documents:user:'.$request->user()->id);
```

Ajoutez la même ligne juste après `$document->update($data);` dans `update`, puis juste après la fin de `DB::transaction(...)` dans `destroy`, avant `return response()->noContent();`. Elle ne doit pas se trouver dans la transaction de suppression.

L'ajout d'un PDF modifie également le document. Dans `app/Http/Controllers/DocumentFileController.php`, ajoutez :

```
use Illuminate\Support\Facades\Cache;
```

Remplacez uniquement la méthode `store` de ce contrôleur par le code ci-dessous ; conservez la méthode `download` existante :

```
    public function store(DocumentFileRequest $request, string $document): JsonResponse
    {
        $path = null;

        try {
            $response = DB::transaction(function () use ($request, $document, &$path): JsonResponse {
                $record = $request->user()->documents()->lockForUpdate()->findOrFail($document);

                if ($record->file_path !== null) {
                    return response()->json(['message' => 'Ce document possède déjà un fichier.'], 409);
                }

                $path = $request->file('file')->store('documents', 'local');
                if ($path === false) {
                    abort(500, 'Impossible de stocker le fichier.');
                }

                $record->file_path = $path;
                $record->save();

                return response()->json(['message' => 'Fichier enregistré.'], 201);
            });
        } catch (Throwable $exception) {
            if (is_string($path)) {
                Storage::disk('local')->delete($path);
            }

            throw $exception;
        }

        if ($response->getStatusCode() === 201) {
```

```
            Cache::forget('documents:user:'.$request->user()->id);
        }

        return $response;
    }
```

La clé n'est supprimée qu'après un envoi réussi : un conflit **409** ne modifie pas le document. La transaction de suppression du document et son nettoyage du fichier privé restent en place.

9. Tester modification et suppression

1. Dans Postman, utilisez à nouveau le token d'Amina. Envoyez `GET /api/documents`, puis la commande `--scan` de l'étape 7. **À vérifier :** sa clé est présente.
2. Choisissez **PUT** pour `http://localhost:8088/api/documents/ID`, en remplaçant `ID` par celui du document créé. Dans **Body > raw > JSON**, envoyez :

```
{ "title": "Document Amina modifié", "category": "Projet" }
```

À vérifier : réponse **200**. Relancez `--scan` : la clé d'Amina a disparu. La clé de Karim, si elle n'a pas expiré, est conservée.

3. Renvoyez le GET avec le token d'Amina. **À vérifier :** le titre modifié apparaît et la clé d'Amina est recréée. Son TTL est à nouveau proche de 300 secondes.
4. Avec le même token, envoyez **DELETE** sur `http://localhost:8088/api/documents/ID`. **À vérifier :** réponse **204** et clé d'Amina absente. Renvoyez le GET : le document supprimé n'apparaît plus et la liste est de nouveau mise en cache.
5. Pour vérifier la création, avec une liste déjà en cache, créez un autre document par POST : la clé doit disparaître ; le GET suivant affiche le nouveau document.

Attention : ces opérations modifient réellement les documents de la **base de formation**. Ne supprimez que les documents fictifs que vous venez de créer. N'utilisez pas `FLUSHALL`, `FLUSHDB` ni `docker compose down -v`.

À vous

Expliquez pourquoi on supprime la copie en cache **après** la réussite de l'écriture dans PostgreSQL. Que se passerait-il si l'on oubliait l'invalidation et que la durée de vie était de cinq minutes ?

Vérifications et difficultés des parties 1 à 3

Situation	Vérification
Pas de `PONG`	`docker compose ps`, nom du service et indentation YAML.
`Class Predis... not found`	Vérifiez `composer show predis/predis` dans `app` et l'installation dans son volume.
Connexion refusée à Valkey	Vérifiez `REDIS_HOST=valkey`, l'état de `valkey`, `config:clear` et le redémarrage de `app`.
Aucune clé	Vérifiez un GET authentifié, `CACHE_STORE=redis`, la base logique `-n 1` et le `user_id`.
Liste d'un autre utilisateur	Arrêtez l'essai et vérifiez immédiatement le token et la clé contenant `user_id`.
Ancien titre après PUT	Vérifiez `Cache::forget` après `update` et utilisez le même utilisateur pour les requêtes.

Situation	Vérification
401 ou 429	Reconnectez-vous si le token a expiré ; après trop de connexions, patientez une minute.

La durée de vie limite une copie ancienne mais ne remplace pas l'invalidation. Cet exemple explique le principe dans une application locale ; des écritures et lectures simultanées peuvent nécessiter une stratégie de cohérence plus élaborée en production.

Comparer avec la correction

Après avoir enregistré votre travail et récupéré la branche si nécessaire, comparez sans changer de branche :

```
git diff jour4-valkey-cache-documents -- compose.yaml .env.example composer.json
app/Http/Controllers/DocumentController.php app/Http/Controllers/DocumentFileController.php
```

La branche de correction contient aussi des tests automatiques. Ne changez pas de branche sans avoir d'abord sauvegardé vos fichiers et vérifié `git status`.

Partie 4 : limiter les requêtes de la liste — 25 min

Comprendre le Rate Limiting

Le **Rate Limiting** limite le nombre de requêtes acceptées pendant une durée donnée. Dans cet exercice, chaque utilisateur connecté peut demander la liste de ses documents **5 fois par minute**. Ce petit nombre sert uniquement à voir le résultat facilement.

Par exemple, si le compteur d'Amina est vide et qu'elle envoie rapidement six demandes pour `GET /api/documents` :

Demande	Résultat
De la 1re à la 5e	Laravel traite la requête ; la liste répond normalement **200 OK**.
6e avant la fin du délai	Laravel répond **429 Too Many Requests** : il faut attendre avant de réessayer.
Après le délai	Amina peut demander sa liste à nouveau.

Pour savoir combien de demandes ont été faites, Laravel utilise un **compteur temporaire**. À chaque demande, il consulte ce compteur : si la limite n'est pas atteinte, il traite la requête ; sinon, il répond 429. Dans les en-têtes de la réponse 429, `Retry-After` indique le nombre de secondes à attendre.

Ici, chaque compteur est lié à l'**identifiant de l'utilisateur** : si Amina atteint sa limite, Karim garde son propre quota. Une limite **par adresse IP** fonctionnerait autrement : plusieurs personnes utilisant la même adresse pourraient partager le même compteur.

Dans notre TD, Laravel utilise **Valkey** pour conserver les compteurs. Un autre stockage de cache peut aussi être configuré ; Valkey n'est donc pas obligatoire pour comprendre le Rate Limiting. Le compteur est distinct du cache de la liste des documents : le premier compte les requêtes, le second conserve temporairement leur résultat.

Cette limite aide à réduire les abus et la surcharge. L'authentification et le contrôle du propriétaire restent nécessaires : un quota disponible ne donne pas accès aux documents d'une autre personne.

10. Préparer l’exercice

Continuez avec votre travail des trois premières parties ou utilisez leur correction, la branche Laravel `jour4-valkey-cache-documents`. Le présent guide se trouve sur cette branche. La correction de cette quatrième partie sera sur `jour4-rate-limiting-documents`.

Avant tout changement de branche, enregistrez et sauvegardez votre travail, puis vérifiez `git status` : ne forcez pas un changement si Git le refuse. Le projet doit déjà utiliser Predis et Valkey comme dans la partie 1.

```
git branch --show-current
docker compose ps
docker compose exec valkey valkey-cli ping
docker compose exec app php artisan config:show cache.default
```

À vérifier : Valkey répond `PONG` et le cache par défaut est `redis`. Dans ce projet, les compteurs de limitation utiliseront donc Valkey. Si `CACHE_STORE` est resté sur `file`, reprenez la configuration de la partie 1 et redémarrez `app`. Ne recréez ni `.env` ni `APP_KEY` et ne relancez ni migration ni seeder.

La route `POST /api/login` possède déjà une limite distincte de **10 tentatives par minute** depuis le jour 3. Nous allons ajouter un autre compteur, sur **la liste `GET /api/documents` seulement**, avec un maximum de **5 demandes par minute et par utilisateur**. Cinq est un petit nombre choisi pour observer facilement le résultat en classe.

11. Définir la limite par utilisateur

Dans VS Code, ouvrez `app/Providers/AppServiceProvider.php`. Ajoutez ces trois lignes parmi les imports existants, sous `namespace App\Providers;` :

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\RateLimiter;
```

Conservez l’import `ServiceProvider` déjà présent. Dans ce même fichier, remplacez **uniquement la méthode `boot`** par :

```
    public function boot(): void
    {
        RateLimiter::for('documents', function (Request $request): Limit {
            return Limit::perMinute(5)->by('documents:'.$request->user()->id);
        });
    }
```

Enregistrez. `documents` est le nom du limiteur ; `perMinute(5)` donne cinq demandes par minute ; `by(...)` crée un compteur distinct pour chaque identifiant utilisateur. Dans cet exercice, la route qui utilisera ce limiteur est protégée par `auth:sanctum` : Laravel identifie d’abord la personne.

À vous

Si Amina et Karim utilisent le même ordinateur, doivent-ils partager les cinq demandes ? Expliquez votre réponse à partir de `by(...)` avant de continuer.

12. Appliquer le limiteur à la liste

Ouvrez `routes/api.php`. Dans le groupe `Route::middleware('auth:sanctum')`, recherchez :

```
    Route::get('/documents', [DocumentController::class, 'index']);
```

Remplacez **cette ligne seulement** par :

```
    Route::get('/documents', [DocumentController::class, 'index'])->middleware('throttle:documents');
```

Enregistrez. `throttle:documents` demande à Laravel d'utiliser le limiteur défini à l'étape précédente. Le groupe `auth:sanctum` reste en place ; la route `/api/login` conserve sa limite séparée.

Dans le terminal du projet Laravel :

```
docker compose exec app php artisan route:list -v --path=api/documents
```

À vérifier : sous `GET api/documents`, la sortie affiche les classes de middleware `Authenticate:sanctum` et `ThrottleRequests:documents` (avec leur nom complet). Si une ancienne configuration de routes avait été mise en cache et que la modification n'apparaît pas, demandez au formateur avant de lancer `docker compose exec app php artisan route:clear`. Cette commande retire le fichier de routes compilées ; elle ne vide pas les compteurs Valkey ni PostgreSQL.

13. Observer 200, puis 429 dans Postman

1. Mettez en pause les autres onglets ou applications qui interrogent la liste avec le compte d'Amina, y compris l'interface web : leurs requêtes comptent aussi.
2. Connectez Amina avec `POST http://localhost:8088/api/login`, **Body > raw > JSON**, comme au jour 3. Copiez le token reçu. Ne le partagez pas.
3. Ouvrez un autre onglet. Choisissez **GET**, saisissez `http://localhost:8088/api/documents`, ajoutez `Accept: application/json` dans **Headers**, puis choisissez **Authorization > Bearer Token** et collez le token d'Amina.
4. Cliquez sur **Send** cinq fois en comptant les requêtes. Puis cliquez une sixième fois.

À vérifier : si son compteur était vierge au départ, les cinq premières réponses sont **200 OK** et la sixième est **429 Too Many Requests**. Dans les **Headers de la réponse**, repérez `X-RateLimit-Limit` et `X-RateLimit-Remaining`. Sur la réponse 429, `Retry-After` indique le nombre de secondes à attendre avant de réessayer.

Si vous obtenez 429 avant la sixième demande, Amina a probablement déjà utilisé une partie de son quota. Attendez le délai indiqué dans `Retry-After`, vérifiez qu'aucune autre application ne demande la liste avec son compte, puis recommencez à compter. **Ne videz pas tout le cache Valkey** pour cet exercice.

5. Après le délai indiqué, cliquez à nouveau sur **Send**. **À vérifier :** la réponse redevient **200**. Le compteur est lié à l'utilisateur, pas à la valeur de son token : demander un nouveau token pour Amina ne donne pas un nouveau quota. L'interface du TD précédent affiche encore un message générique si la liste reçoit 429 : utilisez Postman pour voir le code et le délai, puis patientez avant de recharger l'interface.

14. Vérifier un autre compte et la portée de la limite

1. Recommencez rapidement avec Amina jusqu'à obtenir **429** sur la liste. Gardez cet onglet.
2. Dans l'onglet de connexion Postman, connectez Karim et copiez **son** token. Attention à la limite séparée de connexion : après trop de tentatives sur `/api/login`, attendez une minute.
3. Ouvrez un nouvel onglet **GET /api/documents**, ajoutez `Accept: application/json` et le token de Karim dans **Authorization > Bearer Token**. Cliquez sur **Send**.

À vérifier : Karim reçoit **200** si son propre quota est disponible, même si Amina a atteint sa limite. Sa liste ne contient pas les documents d'Amina. L'authentification et le contrôle du propriétaire continuent de s'appliquer.

La nouvelle limite concerne la **liste** `GET /api/documents`. Laravel accepte aussi `HEAD` sur cette même route, avec le même limiteur. Les routes de lecture d'un document précis, de création, de modification, de suppression et de fichier ne sont pas limitées par ce nouveau compteur. La limite de connexion reste une protection distincte. Le rate limiting réduit certains abus, mais ne constitue pas à lui seul une protection complète contre toutes les attaques.

À vous

1. Que signifie le statut **429** ?
2. Pourquoi la clé du compteur contient-elle l'identifiant utilisateur ?
3. Quelle différence voyez-vous entre le cache de la liste et le compteur de requêtes ?

15. Comparer avec la correction

Une fois votre travail sauvegardé, comparez sans changer de branche, après récupération éventuelle de la branche de correction :

```
git diff jour4-rate-limiting-documents -- app/Providers/AppServiceProvider.php routes/api.php
```

La branche de correction contient également un test automatisé. Il vérifie cinq réponses autorisées, une réponse 429, la séparation entre utilisateurs et le retour après le délai. Vous n'avez pas besoin de créer ce fichier pour terminer le TD.

Difficultés de la partie 4

Situation	Vérification
Erreur de limiteur introuvable	Vérifiez le nom `documents` dans `AppServiceProvider` et `throttle:documents` dans la route.

Situation	Vérification
Erreur sur l'utilisateur	Vérifiez que la route reste dans le groupe `auth:sanctum` et qu'un token valide est envoyé.
429 dès le début	Consultez `Retry-After`, attendez le délai et suspendez les autres clients utilisant le même compte.
Karim reçoit aussi 429	Vérifiez son token et attendez son propre délai si son quota est déjà utilisé.
Le compteur ne réagit pas après modification	Vérifiez la route, le cache par défaut et la configuration Laravel avec le formateur.
Impossible de se connecter	La limite de `/api/login` est distincte ; attendez si elle a été atteinte et vérifiez les identifiants.

Vérifications et bilan du jour 4 — 10 min

Avec le formateur, vérifiez les points suivants, en respectant les délais de limitation :

- Sans token, la liste protégée répond **401**. Avec un token valide et un quota disponible, elle répond **200**.
- Amina et Karim voient chacun leurs propres documents. Un PDF privé est téléchargeable uniquement par son propriétaire.
- Valkey répond **PONG**. La liste est mise en cache avec une clé par utilisateur et une durée de vie ; une modification retire la bonne clé.
- Après cinq demandes autorisées sur la liste, la suivante répond **429**. Un autre utilisateur conserve son propre quota et `Retry-After` indique quand réessayer.

Pour conclure à l'oral : quel est le rôle de Laravel, de PostgreSQL et de Valkey ? Quelle différence faites-vous entre **authentifier**, **autoriser**, **mettre en cache** et **limiter les requêtes** ?